

Relational Reasoning Over Spatial-Temporal Graphs for Video Summarization

Wencheng Zhu, Yucheng Han, Jiwen Lu^{ID}, *Senior Member, IEEE*, and Jie Zhou^{ID}, *Senior Member, IEEE*

Abstract—In this paper, we propose a dynamic graph modeling approach to learn spatial-temporal representations for video summarization. Most existing video summarization methods extract image-level features with ImageNet pre-trained deep models. Differently, our method exploits object-level and relation-level information to capture spatial-temporal dependencies. Specifically, our method builds spatial graphs on the detected object proposals. Then, we construct a temporal graph by using the aggregated representations of spatial graphs. Afterward, we perform relational reasoning over spatial and temporal graphs with graph convolutional networks and extract spatial-temporal representations for importance score prediction and key shot selection. To eliminate relation clutters caused by densely connected nodes, we further design a self-attention edge pooling module, which disregards meaningless relations of graphs. We conduct extensive experiments on two popular benchmarks, including the SumMe and TVSum datasets. Experimental results demonstrate that the proposed method achieves superior performance against state-of-the-art video summarization methods.

Index Terms—Video summarization, spatial-temporal representation, self-attention, graph pooling, relational reasoning.

I. INTRODUCTION

VIDEO summarization has been a significant research topic in computer vision over the past few years [1]–[4], especially when ever-growing online videos can be easily accessible and available. There have been many practical applications in video summarization, such as movie trailers [5], [6], sports video skimming [7], [8], and video surveillance analysis [9], [10]. Generally, the objective of video summarization is to identify representative video frames that convey the most important and relevant information in a video sequence [11], [12]. Existing video summarization methods can be broadly categorized into two classes: 1) static and 2) dynamic approaches. Generally, static approaches select several keyframes as video summaries, while dynamic approaches create video summaries by choosing video segments with consecutive frames [8], [13]. Since video segments

are easy for users to understand, the recently proposed methods mainly fall into dynamic video summarization. Dynamic video summarization methods are usually composed of three steps: 1) feature extraction, 2) frame-level importance score prediction, and 3) key shot selection [14].

Recently, numerous single and multiple view video summarization methods have been proposed [15]–[17] and these methods have made encouraging progress. Especially, multi-view video summarization is a more challenging task, which summarizes videos from a camera network [18]. In this paper, we concentrate on the single-view video summarization topic. The latest single-view video summarization methods are almost deep learning based approaches, which can be roughly classified into three categories: 1) RNN (Recurrent Neural Networks) based [19], [20], 2) adversarial learning based [21], [22], and 3) attention based [23], [24]. RNN based methods have reported superior results, however, these methods are poor at handling video sequences of long duration [25]. Adversarial learning based methods cannot learn from annotated data. Moreover, their performance heavily relies on the introduced video generation task that requires costly computation resources [22]. Attention based methods achieve remarkable performance with the attention mechanism [23]. Existing deep learning based methods mainly extract image-level representations for the importance score prediction, however, these methods are not capable enough to reveal dynamic visual contexts and relationships inside video frames.

To address the above issues, we propose a novel video summarization method called *RR-STG*, which establishes spatial-temporal graphs to model the detected object proposals and their relations in a video sequence. Fig. 1 illustrates the basic idea of the proposed method. Our method is primarily motivated by insights that video summarization pays more attention to key objects and frames as well as their relationships for visual understanding. However, there is too much meaningless information among these temporal and spatial instances.

To focus on key objects and frames, we build a spatial graph of object proposals in each frame and a temporal graph of the aggregated spatial graphs, where the node representations of these spatial and temporal graphs are feature vectors of object proposals and spatial graph representations, respectively. To be specific, for each video frame, we extract its object proposals by using the region proposal networks [26] and represent these object proposals and their relations via a spatial graph. To further capture temporal dependencies among different video frames, we first aggregate feature vectors of object

Manuscript received September 17, 2020; revised April 22, 2021 and October 19, 2021; accepted March 24, 2022. Date of publication April 6, 2022; date of current version April 12, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 62125603 and Grant U1813218, in part by a grant from the Beijing Academy of Artificial Intelligence (BAAI), and in part by the Shuimu Tsinghua Scholar Program (2021SM012). The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Ghassan AlRegib. (*Corresponding author: Jiwen Lu.*)

The authors are with the Beijing National Research Center for Information Science and Technology (BNRist) and the Department of Automation, Tsinghua University, Beijing 100084, China (e-mail: wenchengzhu@mail.tsinghua.edu.cn; yucheng002@e.ntu.edu.sg; lujiwen@tsinghua.edu.cn; jzhou@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TIP.2022.3163855

1941-0042 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

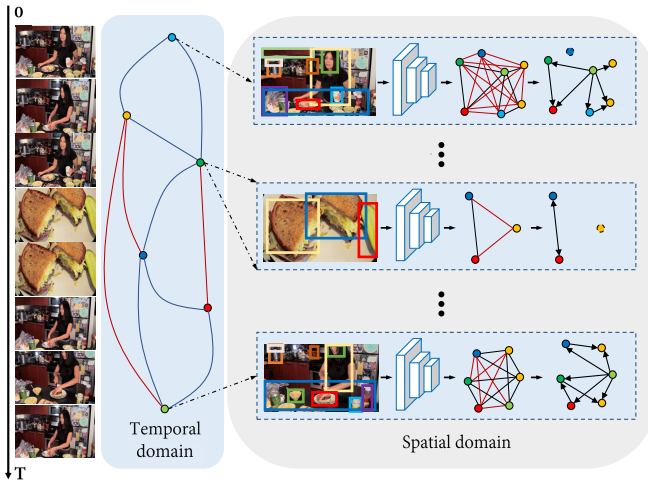


Fig. 1. The core idea of the proposed method. Our method consists of one temporal graph and T spatial graphs. For each node in the temporal graph, there is a spatial graph that represents object-level and relation-level dependencies. To be specific, we pay more attention to relations among object proposals and conduct relational reasoning to update spatial graphs. For example, as the kettle is unrelated to the other objects, we drop this node and remove all its linkings with the other object proposals. Furthermore, we also perform relational reasoning to capture temporal dependencies within a video sequence. Note that red lines will be removed from graphs.

proposals as spatial graph representations. Then, we model spatial graph representations via a temporal graph. Afterward, we employ graph convolutional networks [27] to conduct message passing over spatial and temporal graphs.

Furthermore, to handle redundancy connections in graphs, we further design a self-attention edge pooling module named *SAEPool*, which can adaptively remove graph edges in both spatial and temporal domains. The proposed graph pooling layer leverages not only the second-order similarity information between different nodes but also the graph topology and graph representations to select graph edges. Having extracted spatial-temporal representations of video frames, we estimate their frame-level importance scores. Additionally, we conduct video shot detection and compute the shot-level importance scores. Finally, the key shot selection is performed to produce video summaries. We conduct experiments on two widely-used video summarization datasets, i.e., SumMe and TVSum. Experimental results clearly validate the effectiveness of the proposed method.

The main contributions of the proposed method are summarized as four-fold:

- Different from conventional works on image-level representations, our method extracts both the object-level and relation-level information in video frames.
- We propose a dynamic graph modeling approach for video summarization, which employs graph convolutional networks and edge pooling layers to conduct message passing over spatial and temporal graphs.
- Moreover, we design the self-attention edge pooling module to handle the relation clutter issue.
- We conduct experiments on two standard video summarization datasets. Experimental results demonstrate the effectiveness of the proposed method.

The remainder of this paper is organized as follows. Section II briefly reviews the related literature. Section III details the proposed method. Section IV compares our method on two benchmarks and visualizes example results. Section V concludes this paper.

II. RELATED WORK

In this section, we briefly outline the following three related topics: 1) video summarization [28]–[30], 2) visual reasoning [31], [32], and 3) graph pooling [33].

A. Video Summarization

Over the past few years, deep learning based methods have brought significant progress in video summarization [34]–[37]. Existing deep learning methods can be mainly divided into three categories: RNN based, adversarial learning based, and attention based. Methods in the first category employed RNN to capture long-range temporal dependencies over video sequences. For example, Zhang *et al.* [38] extracted long-range information of video frames by using LSTM. Zhao *et al.* [20], [39] designed the hierarchical RNN architecture to exploit the underlying hierarchical structure in video sequences. Zhang *et al.* [40] formulated video summarization as a sequence-to-sequence learning problem. Zhou *et al.* [41] adopted the deep reinforcement learning strategy to select representative frames with largest rewards. Jung *et al.* [42] constructed a novel two-stream architecture to learn both local and global temporal information. Huang *et al.* [17] extracted multi-stage spatio-temporal representations for video summarization. Unlike RNN based methods that are challenging to extract temporal information of long sequences, our method can solve this problem by using graph layers. Methods in the second category exploited adversarial learning to guarantee consistency between generated videos and original videos. For example, Mahasseni *et al.* [22] applied a variational auto-encoder and a generative adversarial network to reconstruct and discriminate original videos, respectively. Rochan and Wang [43] proposed an adversarial video summarization approach for unpaired data. Yuan *et al.* [44] developed a cycle-consistent adversarial LSTM networks for unsupervised video summarization. Zhang *et al.* [21] presented a three-player discriminator to guarantee completeness and compactness of generated summaries. Most adversarial learning based methods are unsupervised approaches. Differently, our method is a supervised approach and can learn from annotated data. Methods in the third category adopted the attention mechanism to predict the importance scores of video frames. For example, Zhong *et al.* [23] proposed attentive encoder-decoder networks (AVS) to place different attention on video frames. Fajtl *et al.* [25] employed the self-attention mechanism for attention weight calculation. Liu *et al.* [24] employed hierarchical multi-attention to uncover video structures. Xiao *et al.* [45] learned visual information of each shot with local self-attention. Unlike attention based methods that exploit image-level representations, our method captures dynamic object-level and relation-level information. There are some other methods. For example, Zhong *et al.* [46]

exploited spatio-temporal information for imbalanced video summarization with a two-stream deep architecture. Differently, our method extracts spatial-temporal representations via graph modeling. Wu *et al.* [37] firstly adopted graph networks to handle multi-video summarization and proposed dynamic graphs with the bagging and penalty loss strategies for the class imbalance issue and a diversity regularization for diverse summaries. This method built a shot graph of multiple videos, however, our method is a single-video summarization approach with spatial and temporal graphs, where nodes are object proposals and spatial representations.

B. Visual Reasoning

In recent years, there have been a variety of studies on visual reasoning [31], [32]. Earlier works mainly focused on the relational reasoning of static images for visual question answering. For example, Battaglia *et al.* [47] introduced the Interaction Networks to reason about objects and relations in complex physical systems. Santoro *et al.* [48] proposed the Relation Networks to solve relational reasoning problems. Besides, scene graph methods have been developed to classify objects and relations. For example, Tang *et al.* [49] learned dynamic tree structures of visual contexts for relation context encoding. Zellers *et al.* [50] proposed a stacked motif network for scene graph parsing. Chen *et al.* [51] proposed an iterative visual reasoning framework that iteratively conducted convolution-based and graph-based information passing. Shi *et al.* [52] developed an explainable and explicit visual reasoning method over scene graphs. Unlike static image reasoning methods and scene graph methods that only exploit the spatial structure, our method takes both spatial and temporal information into consideration. Then, many graph modeling works have been proposed to perform reasoning in videos [53], [54]. For example, Wang *et al.* [55] conducted reasoning on space-time region graphs via graph convolutional networks [27]. Zhuo *et al.* [56] developed an attribute-based action reasoning model and a relationship-based action reasoning model over a video graph. Tsai *et al.* [57] applied Conditional Random Field on a spatio-temporal graph that captured statistical dependencies between spatial and temporal relational entities. Jang *et al.* [58] proposed a dual-LSTM based approach with spatial and temporal attention, which conducted spatio-temporal reasoning from videos. Different from conventional graph modeling methods, our method removes redundancy information among graphs.

C. Graph Pooling

There have been numerous graph pooling methods proposed to solve the generalization problem of complex graphs [33]. Existing graph pooling methods can be broadly grouped into three categories: 1) topology based, 2) global based, and 3) hierarchical pooling methods. For the first category, the graph coarsening algorithms were usually adopted to obtain fixed cluster assignments. Typical methods in this category are graphs with fast localized spectral filtering [59], localized graph convolutional filtering [60], and dynamic edge-conditioned filters [61]. For the second category, graph

features were extracted for feature based cluster assignments. For example, Gao and Ji [62] developed the graph U-Nets that adopted a top-k node selection strategy to pool nodes. Lee *et al.* [63] took both the graph topology and node features into consideration for graph node scoring. Zhang *et al.* [64] sorted and pooled the vertex features with the proposed SortPooling layer. Yuan *et al.* [64] applied conditional random fields to learn high-order structural relationships among graph nodes. In the third category, hierarchical graph pooling methods extracted hierarchical representations for structure based cluster assignments. For example, Ying *et al.* [65] iteratively learned a soft clustering assignment for nodes in each GNN layer. Cangea *et al.* [66] adaptively selected top-ranked nodes with the higher projection scores in each pooling layer. These methods are mostly node pooling approaches, our method is an edge pooling approach.

III. PROPOSED APPROACH

In this section, we first provide an overview of the proposed method. Then, we detail the spatial-temporal graphs, the self-attention edge pooling module, and key shot selection.

A. Overview

Fig. 2 illustrates the overall framework of the proposed RR-STG, which consists of feature extraction of object proposals, spatial and temporal representation learning for importance score prediction, and key shot selection steps. Specifically, given a video sequence with T frames, our method builds a spatial-temporal graph that is composed of spatial graphs $\mathcal{G}_s = (\mathcal{V}_s, \mathcal{E}_s)$, $s \in \{1, 2, \dots, T\}$ of object proposals and a temporal graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ of video frames. The node representations $\mathbf{V}_s = [\mathbf{v}_{s1}, \mathbf{v}_{s2}, \dots, \mathbf{v}_{sn}] \in \mathbb{R}^{n \times d_o}$ of the s^{th} spatial graph are extracted from object boundary boxes $\mathbf{B}_s = \{\mathbf{b}_{s1}, \mathbf{b}_{s2}, \dots, \mathbf{b}_{sn}\}$, where d_o is the dimension of feature vectors in spatial graphs, $\mathbf{B}_s$ represents the set of boundary boxes, and n is the selected number of boundary boxes in each frame. The node representations $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_T] \in \mathbb{R}^{T \times d_t}$ of the temporal graph are defined as the aggregated representations of object proposals in a spatial graph, where d_t is the dimension of temporal features. We employ graph convolutional layers (GCN) to perform relational reasoning and learn spatial-temporal representations for importance score prediction. Finally, we conduct key shot selection with shot segmentation and the knapsack algorithm to create summaries.

B. Spatial-Temporal Graphs

Most existing graph based video modeling approaches represent a video sequence as a graph of all object proposals [55]–[57]. However, these approaches are not suitable to tackle the video summarization task. One reason is that video summarization requires frame-level features for an importance score prediction, but these methods only provide video-level representations. Another reason is that representing a whole video sequence as a graph cannot directly reflect dynamical changes of visual contexts, especially when videos are captured in complex scenes. Furthermore, efficient and

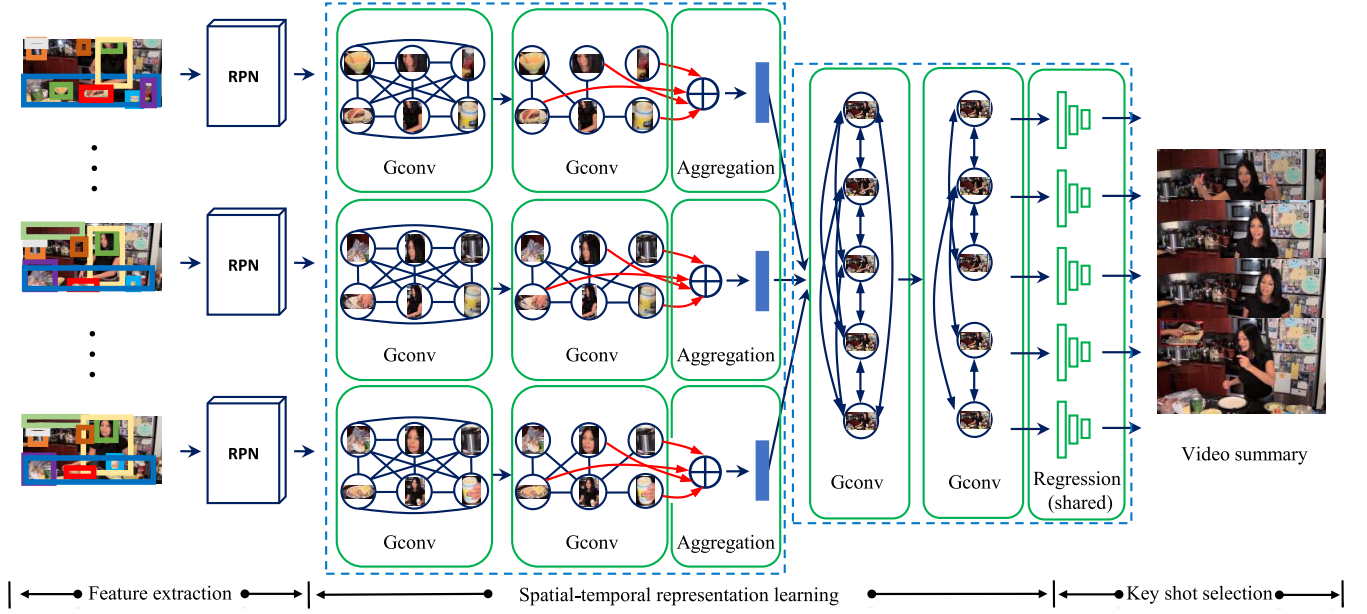


Fig. 2. The illustration of the proposed RR-STG. Our method mainly includes three steps: 1) feature extraction of object proposals, 2) spatial and temporal representation learning for frame-level importance score prediction, and 3) key shot selection. We first detect object proposals in video frames and apply graph convolution layers (Gconv) to perform message passing for spatial and temporal representations. Then, we estimate the importance scores of video frames with the regression module. Finally, we conduct key shot selection to produce summaries.

effective message passing in such a large graph is challenging. To cope with these issues, we build separable spatial-temporal graphs.

Spatial Graph: We adopt Faster RCNN [26] with a ResNet-50-FPN backbone to extract bounding boxes that capture object-level spatial information of video frames. Then, we filter out these candidate bounding boxes via the non-maximal suppression algorithm (NMS) [26] and set its IoU threshold as 0.5. For each frame, we select n object proposals with top-ranked scores. Next, we feed features of bounding boxes into a RoIAlign layer [67] and align them into 1024 dimensions. We extract features of bounding boxes other than the feature of the whole image as object-level features have been proven to be effective on high-level computer vision tasks, such as video classification [55], [57], scene graph parsing [50], and segmentation [68].

Moreover, to represent visual content in a video frame, we model the video frame as a spatial graph with nodes being object proposals. We conduct message passing among adjacent nodes in the s^{th} spatial graph as,

$$\mathbf{m}_{si}^{(k)} = \sum_{j:(i,j) \in \mathcal{E}_s} f_{msg}(\mathbf{v}_{sj}^{(k-1)}), \quad (1)$$

$$\mathbf{n}_{si}^{(k)} = f_{node}(\mathbf{v}_{si}^{(k-1)}), \quad (2)$$

$$\mathbf{v}_{si}^{(k)} = f_{update}(\mathbf{m}_{si}^{(k)}, \mathbf{n}_{si}^{(k)}), \quad (3)$$

where $\mathbf{m}_{si}^{(k)}$, $\mathbf{n}_{si}^{(k)}$, and $\mathbf{v}_{si}^{(k)}$ are the message vector, the embedded feature, and the feature of the i^{th} node in the k^{th} layer. f_{msg} , f_{node} , and f_{update} are the message passing function, the node embedded function, and the node update function. Eq. (1) indicates the passing message from adjacent nodes, Eq. (2)

represents the self-passing message, and Eq. (3) combines information to update the i^{th} node feature. f_{msg} , f_{node} , and f_{update} can be implemented by using differentiable mappings. We adopt the same neural networks for f_{msg} and f_{node} , while f_{update} is defined as a summation function. Formally, the graph convolutional layer can well represent the message passing scheme as,

$$\mathbf{V}_s^{(k)} = \sigma \left(\tilde{\mathbf{D}}_s^{(k-1)^{-\frac{1}{2}}} \tilde{\mathbf{A}}_s^{(k-1)} \tilde{\mathbf{D}}_s^{(k-1)^{-\frac{1}{2}}} \mathbf{V}_s^{(k-1)} \Theta_s \right), \quad (4)$$

where σ represents an activation function, $\tilde{\mathbf{A}}_s^{(k-1)} = \mathbf{A}_s^{(k-1)} + \mathbf{I}_n \in \mathbb{R}^{n \times n}$ is the adjacent matrix with self-connections, $\tilde{\mathbf{D}}_s^{(k-1)} \in \mathbb{R}^{n \times n}$ is the degree matrix of $\tilde{\mathbf{A}}_s^{(k-1)}$, $\mathbf{A}_s^{(k-1)}$ represents pairwise similarities computed by using input features $\mathbf{V}_s^{(k-1)}$, and $\Theta_s \in \mathbb{R}^{d_o \times d_o}$ is denoted as parameters in the graph convolutional layer. We learn the updated node representations after several graph convolutional layers. These node representations are passed into an average pooling layer and are aggregated as graph representations $\mathbf{v}_s$.

Temporal Graph: We further represent the spatial graph representations $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_T]$ as a temporal graph. The same as spatial graphs, we conduct message passing among adjacent video frames as,

$$\mathbf{m}_i^{(k)} = \sum_{j:(i,j) \in \mathcal{E}_t} f_{msg}(\mathbf{v}_j^{(k-1)}), \quad (5)$$

$$\mathbf{n}_i^{(k)} = f_{node}(\mathbf{v}_i^{(k-1)}), \quad (6)$$

$$\mathbf{v}_i^{(k)} = f_{update}(\mathbf{m}_i^{(k)}, \mathbf{n}_i^{(k)}), \quad (7)$$

where $\mathbf{m}_i^{(k)}$, $\mathbf{n}_i^{(k)}$, and $\mathbf{v}_i^{(k)}$ are the message vector, the embedded feature, and the feature of the i^{th} node in the k^{th} layer.

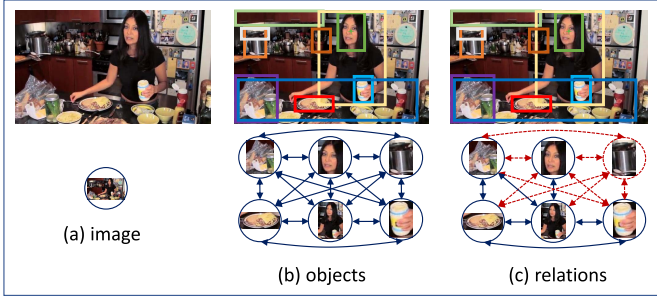


Fig. 3. Comparisons of the frame-level feature extraction strategies. Conventional video summarization methods extract image-level representations by using deep neural networks that are pre-trained on ImageNet. We apply the scene graph scheme and detect object proposals to capture object-level dependencies. However, there are redundancy relations. For example, the kettle is not related to the other objects and the pie has little concern with the woman's face. We need to eliminate unrelated nodes and edges and conduct relational reasoning over simplified spatial graphs.

Likewise, we implement this message passing scheme by using a graph convolutional layer as,

$$\mathbf{V}^{(k)} = \sigma \left(\tilde{\mathbf{D}}^{(k-1)^{-\frac{1}{2}}} \tilde{\mathbf{A}}^{(k-1)} \tilde{\mathbf{D}}^{(k-1)^{-\frac{1}{2}}} \mathbf{V}^{(k-1)} \Theta \right), \quad (8)$$

where $\tilde{\mathbf{A}}^{(k-1)} = \mathbf{A}^{(k-1)} + \mathbf{I}_T \in \mathbb{R}^{T \times T}$ represents the adjacent matrix with self-connections, $\tilde{\mathbf{D}}^{(k-1)} \in \mathbb{R}^{T \times T}$ is the degree matrix, and $\Theta \in \mathbb{R}^{d_t \times d_t}$ represents parameters in the temporal graph convolutional layer. Then, we obtain the spatial-temporal representations $\mathbf{V}^{(k)}$ of video frames via relational reasoning over spatial and temporal graphs.

Finally, we feed the learned spatial-temporal representations into a regression module $g(\cdot)$ to estimate importance scores. We formulate the objective as,

$$\mathcal{L} = \|\mathbf{g}(\mathbf{V}^{(k)}; \tilde{\theta}) - \mathbf{y}\|_2^2, \quad (9)$$

where $\mathbf{y}$ represents the ground truth frame-level importance scores, $\tilde{\theta}$ represents parameters in the regression module. We use the mean-square error loss as the loss function $\mathcal{L}$. The regression module consists of three linear layers, followed by tanh and dropout(0.5), tanh, and sigmoid layers, respectively.

Discussion: Fig. 3 provides comparisons of different frame-level feature extraction strategies. We observe that conventional video summarization methods employ deep models such as GoogLeNet [69] and ResNet [70] pre-trained on ImageNet to extract frame-level representations. However, these representations disregard fine-grained object-level information. To address this issue, we extract object proposals and model the observed object instances and their relations as nodes and edges. We further reason over this spatial graph for its graph representation. However, we also observe that due to fully connected linkings among object proposals, there are many meaningless relations in graphs and these unimportant interplays may cause a severe graph edge saturation problem. Therefore, a graph pooling approach to perform edge removal is necessarily required.

C. Self-Attention Edge Pooling

Existing graph modeling methods suffer from the following drawbacks. On one hand, relations among object proposals

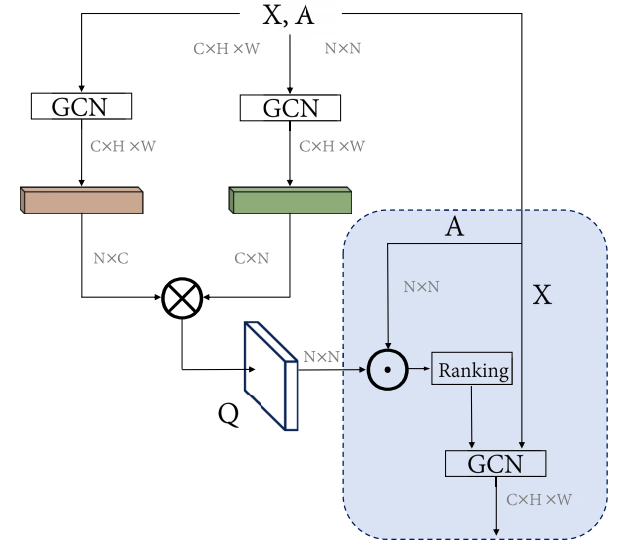


Fig. 4. The architecture of the self-attention edge pooling module. We first embed node representations by employing two graph convolutional layers and obtain the attention map via matrix multiplication between these two embedded representations. Then, a top-rank operator is adopted to select the most informative edges. Finally, the updated representations are outputted by using another graph convolutional layer with inputs of node representations and a renewed adjacent matrix.

are quite sparse [63], [65], [66] but dense connections are commonly adopted in graph modeling [50], leading to relatively expensive computations for message passing. On the other hand, fixed linkings in graphs are not flexible to reveal ever-changing visual cues in video frames a long time. Therefore, numerous graph pooling methods are proposed to conduct relation reduction for graph generalization improvement. However, most graph pooling methods fall into graph coarsening algorithms, which merely take node pooling into consideration. These methods easily produce coarse graphs.

To mitigate this issue, we propose an off-the-shelf self-attention edge pooling module, which performs the subtle edge pooling by exploiting pair-wise similarities among object proposals, the graph topology, and graph representations. The proposed module derives from the observation that aggressively removing nodes from graphs easily changes the original topology structures of graphs. As shown in Fig. 3, for important nodes (e.g., a woman), there are inessential connections (e.g., the woman and the kettle), while for unimportant nodes (e.g., a bread bag), there are several valuable relations (e.g., the bag and the pie). Node pooling methods simply conduct the node-level selection and are incompatible with fine-grained edge removal. However, the proposed self-attention edge pooling module can drop redundant edges. Moreover, unlike self-attention graph pooling [63], our module utilizes second-order information and the original similarity structure. Fig. 4 illustrates the architecture of the proposed **Self-Attention Edge Pooling (SAEPool)** module in detail.

Suppose a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with n nodes and n^2 edges, we denote its node representations as $\mathbf{X} \in \mathbb{R}^{n \times d}$ and corresponding adjacent matrix as $\mathbf{A} \in \mathbb{R}^{n \times n}$. We first embed node features by using two graph convolutional layers, and then

compute their self-attention map $\mathbf{Q} \in \mathbb{R}^{n \times n}$ as,

$$\tau(\mathbf{X}) = \sigma \left(\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{X} \Theta_1 \right), \quad (10)$$

$$\phi(\mathbf{X}) = \sigma \left(\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} \mathbf{X} \Theta_2 \right), \quad (11)$$

$$\mathbf{Q} = \sigma \left(\tau(\mathbf{X}) \phi(\mathbf{X})^\top \right), \quad (12)$$

where $\Theta_1 \in \mathbb{R}^{d \times d}$ and $\Theta_2 \in \mathbb{R}^{d \times d}$ represent parameters in two graph convolutional layers, σ is an activation function, and $\mathbf{D} \in \mathbb{R}^{n \times n}$ is the degree matrix. To be simple, we set these two layers as equal, i.e., $\Theta_1 = \Theta_2$. We conduct experiments to investigate the influence of this setting in the *Experiments* section. The self-attention map $\mathbf{Q}$ reflects pair-wise similarities among node representations.

By applying graph convolutional layers for feature transformation, we can obtain attention maps based on both the graph topology and node features. Having obtained the self-attention map, we select top-ranked $\lfloor kn^2 \rfloor$ edges from n^2 edges in the edge set $\mathcal{E}$. Formally, this process can be written as,

$$\tilde{\mathbf{A}} = \mathbf{A} \odot \mathbf{Q}, \quad (13)$$

$$\tilde{\mathbf{A}}_{\text{idx}} = \text{top-rank} \left(\tilde{\mathbf{A}}, \lfloor kn^2 \rfloor \right), \quad (14)$$

where the hyper-parameter $k \in (0, 1]$ represents the pooling rate of graph edges. The top-rank operator outputs the values and indices of the selected edges. We implement the top-rank operator by using the *torch.topk* function on the flattened $\tilde{\mathbf{A}} \in \mathbb{R}^{n^2}$ as $[\text{values}, \text{indices}] = \text{torch.topk}(\tilde{\mathbf{A}}, \lfloor kn^2 \rfloor)$. Then, we convert values into the matrix format. Since the *torch.topk* function is differentiable, the top-rank operator is also differentiable. $\odot$ is the element-wise dot product, and $\tilde{\mathbf{A}}$ represents the updated adjacent matrix by using the dot product with the self-attention map. We observe that the self-attention edge pooling module chooses top-ranked edges not only considering second-order attention scores but also the similarity structure of the input graph. Finally, we compute the updated node representations $\tilde{\mathbf{X}}$ as,

$$\tilde{\mathbf{A}} = \tilde{\mathbf{A}}_{\text{idx}} \quad (15)$$

$$\tilde{\mathbf{X}} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} \Theta \right), \quad (16)$$

where Θ represents parameters in the graph convolutional layer, and $\tilde{\mathbf{D}}$ is the degree matrix of the pooled adjacent matrix $\tilde{\mathbf{A}}$. For the edge pooling module, we pool graph edges rather than nodes based on three aspects of information, including the graph structure $\mathbf{A}$, graph representation $\mathbf{X}$, and self-attention map $\mathbf{Q}$. It is noteworthy that the node pooling approach [63] is a special case of the edge pooling module, which can be fulfilled by removing all edges related to the corresponding nodes. Moreover, we can simply replace the convention graph convolutional layer with the SAEPool layer to conduct relation reduction in both spatial and temporal domains. Finally, we summarize the training pipeline of the proposed method in **Algorithm 1**.

D. Key Shot Selection

In the training phase, we merely estimate frame-level importance scores without the key shot selection step. In the

Algorithm 1 The Training Pipeline of RR-STG

Input: Training video sequences, the annotated frame-level scores, and video number N_v .

Output: Parameters θ of RR-STG.

Initialize model parameters θ ;

for $epoch \leftarrow 1, 2, \dots, 100$ **do**

for $index \leftarrow 1, 2, \dots, N_v$ **do**

 Sample a video sequence;

for $s \leftarrow 1, 2, \dots, T$ **do**

 // Data-preprocessing:

 Detect bounding boxes $\mathbf{B}_s$;

 Extract feature vectors $\mathbf{V}_s$;

 // Spatial graph

$\mathbf{V}_s^{(k)} = \text{UpdateSpatialNodes}(\mathbf{V}_s)$ with SAEPool by using Eq. (16);

end

$\mathbf{v}_s = \text{AggregateObjectFeatures}(\mathbf{V}_s^{(k)})$;

 // Temporal graph

$\mathbf{V}^{(k)} = \text{UpdateTemporalNodes}(\mathbf{V})$ with SAEPool by using Eq. (16);

$\mathbf{y} = \text{RegressFrameScore}(\mathbf{V}^{(k)})$;

 // Update θ by back propagation

$\mathcal{L} = \text{ComputeLossFunc}(\mathbf{y})$ by using Eq. (9);

$\theta \leftarrow \theta - \eta \partial \mathcal{L} / \partial \theta$;

end

end

Return: The network parameters θ of RR-STG.

testing phase, we first employ the trained model to estimate frame-level importance scores. Then, we exploit the widely-used *Kernel Temporal Segmentation* (KTS) [7] strategy to segment shot boundaries. Once frame-level importance scores and shot segments are obtained, we convert frame-level importance scores into shot-level importance scores. To be specific, we average the frame-level importance scores inside the same shots as the corresponding shot-level importance scores $h_z, z \in \{1, 2, \dots, n_z\}$, where n_z is the number of shots in a video sequence. Finally, we select representative shots for storyline. For a fair comparison, the total length of the selected shots is less than 15% that of the corresponding video sequence, which guarantees that this video sequence is properly summarized. The objective of video summarization can be written as,

$$\max \sum_{z=1}^{n_z} w_z h_z, \quad \text{subject to} \quad \sum_{z=1}^{n_z} w_z l_z \leq 0.15 n_z, \quad (17)$$

where l_z is the length of the z^{th} shot, n_z is the number of video shots, $w_z \in \{0, 1\}$ decides whether or not we select the z^{th} shot in the final summaries. As shown in Eq. (17), we need to solve the standard 0/1 knapsack problem, and we can apply the dynamic programming algorithm to get its optimal solution. Finally, we select shots in sequence where $w_z = 1$ to produce video summaries.

TABLE I
DESCRIPTORS OF VIDEO SUMMARIZATION DATASETS

Dataset	#Video	# User Annotation	Annotation type
SumMe [15]	25	15-18	frame-level scores
TVSum [53]	50	20	frame-level scores
OVP [6]	50	5	keyframes
YouTube [6]	39	5	keyframes

IV. EXPERIMENTS

In this section, we first describe experimental settings. Then, we present experimental results and analyses, and ablation studies. Finally, we intuitively visualize experimental results.

A. Experimental Settings

We detail two standard video summarization datasets, i.e., the SumMe dataset and the TVSum dataset, and the augmented datasets, i.e., the OVP dataset and the YouTube dataset. Then, we elaborate on evaluation metrics, evaluation settings, implementation details, and baselines.

1) *Datasets*: The SumMe dataset [71] consists of 25 raw edited user videos, covering holidays, events, and sports topics. The durations of most videos are about 1 to 6 minutes. The frame-level importance scores are annotated by multiple users. The TVSum dataset [72] contains 50 videos from 10 categories, including making sandwiches, parkour, and grooming an animal. The durations of videos range from 2 to 10 minutes. Moreover, we exploited another two additional video summarization datasets, including OVP and YouTube [8], which were usually used as the augmented datasets. The OVP dataset consists of 50 videos, while the YouTube dataset is composed of 39 videos without cartoon videos. Table I summarizes the key characteristics of four datasets.

Different from the SumMe and TVSum datasets, the OVP and YouTube datasets only provide keyframes. Therefore, we converted keyframes into frame-level importance scores by assigning the important scores of video frames in the same shots with keyframes as 1.0. Furthermore, all videos in these four datasets were uniformly down-sampled from 30 frames per second to 2 frames per second to reduce temporal redundancy.

2) *Evaluation Metrics*: For a fair comparison, we used the conventional evaluation protocol [38], [41] and computed the harmonic mean F-score between ground truth summaries and generated summaries. Given the ground truth summaries $\mathbf{G}$ and the generated summaries $\mathbf{S}$, we first computed precision and recall as,

$$p_i = \frac{|\mathbf{g}_i \cap \mathbf{s}_i|}{|\mathbf{g}_i|}, \quad r_i = \frac{|\mathbf{g}_i \cap \mathbf{s}_i|}{|\mathbf{s}_i|}, \quad (18)$$

where $\mathbf{g}_i$ and $\mathbf{s}_i$ are the i^{th} ground truth and generated summaries, p_i and r_i are the precision and recall of the i^{th} video sequence, respectively. $|\cdot|$ represents the modular operator. Then, we achieved the F-score as,

$$f = \frac{1}{N_v} \sum_{i=1}^{N_v} \frac{2 \times p_i \times r_i}{p_i + r_i}, \quad (19)$$

where N_v is the total number of video sequences. We followed the experimental settings of previous works [38], [41] to deal with video sequences with multiple user annotations. We averaged F-score of different users on TVSum [72] and selected the largest F-score on SumMe [71] as the final results.

3) *Evaluation Settings*: In our experiments, we randomly separated the training dataset into five splits. There were three evaluation settings, including canonical, augmented, and transfer settings, to evaluate the performance of our method. For the canonical setting, we used 80% of videos for training and the remaining 20% for testing. For the augmented setting, we used 80% videos augmented with videos of the other three datasets for training and the rest for testing. In the transfer setting, we tested a given dataset by using the other three datasets for training. Particularly, we set the canonical setting as the default setting. We conducted all experiments five times and reported the mean performance.

4) *Implementation Details*: We employed ResNet-50-FPN that was pre-trained on the Microsoft COCO dataset [26], [81] to extract 1024 dimensional RoI features. For each video frame, we directly employed the pre-trained detector to extract its object proposals and selected the top-ranked 10 object proposals after the non-maximal suppression. Note that we did not fine-tune the detector on video summarization datasets. We calculated the adjacent matrix among object proposals by using the cosine distance. We both stacked two GCN or SAEPool layers over spatial and temporal graphs to capture spatial and temporal information, respectively. We set parameters of two graph convolution layers in Eq. (10) and Eq. (11) to be equal. We used LeakyRelu as the activation function. The pooling rates of temporal and spatial SAE-Pool layers were set as 0.8 and 0.2 on the TVSum dataset, 0.6 and 0.2 on the SumMe dataset. Moreover, the dimension of hidden states in spatial and temporal graphs was set as 1024 and that of spatial-temporal features was set as 512. We employed the regression module, including three linear layers (512, 256, 256) and a dropout layer (0.5), to estimate the frame-level importance scores of video frames. For training details, we trained our model from scratch for 100 epochs with a batch size of 1. We used the Adam optimizer, where the base learning rate was set as 5×10^{-5} and the weight decay was set as 10^{-5} . To alleviate the over-fitting issue, we adopted the early stopping strategy.

5) *Baselines*: We provided comparisons with non-deep and deep learning based methods to evaluate the effectiveness of the proposed method. Non-deep learning based methods include multi-video summarization based on video-mm (Video MMR) [73], exemplar-based subset selection (DPP) [74], quasi real-time summarization (LiveLight) [75], summarizing web videos using titles (TVSum) [72], a general framework for edited video and raw video summarization (ERSUM) [76], and video summarization via multi-view representative selection (MSDS-CC) [77]. Deep learning based methods include video summarization with long short-term memory (vsLSTM and dppLSTM) [38], unsupervised video summarization with adversarial lstm networks (SUM-GAN) [22], user-ranking video summarization (TS-STN) [17], video summarization via semantic attended

TABLE II

COMPARISONS WITH NON-DEEP AND DEEP LEARNING-BASED METHODS ON THE SUMME AND TVSUM DATASETS. MOREOVER, KEY CHARACTERISTICS OF THESE METHODS ARE SUMMARIZED, INCLUDING SHOT DETECTION STRATEGIES, FEATURE EXTRACTION MODELS, LEARNING PARADIGMS, AND MODEL PARAMETERS (MILLION). THE BEST RESULTS ARE MARKED IN BOLD

Method	Shot segmentation	Feature	Learning paradigm	Params	SumMe	TVSum
Video MMR [32]	-	SIFT	Supervised	-	26.6	-
DPP [71]	KTS	AlexNet	Supervised	-	40.9	-
LiveLight [79]	Uniform segmentation	SIFT	Supervised	-	-	46.0
TVSum [53]	Change-point detection	HoG+GIST+SIFT	Unsupervised	-	-	50.0
ERSUM [30]	Uniform segmentation	VGGNet-16	Supervised	-	43.1	59.4
MSDS-CC [36]	KTS	GIST+GoogLeNet	Unsupervised	-	40.6	52.3
vsLSTM [72]	KTS	GoogLeNet	Supervised	2.63	37.6	54.2
dppLSTM [72]	KTS	GoogLeNet	Supervised	2.63	38.6	54.7
SUM-GAN [35]	KTS	GoogLeNet	Unsupervised	295.86	39.1	51.7
TS-STN [19]	KTS	VGGNet-16	Supervised	16.18	46.1	60.0
SASUM [59]	KTS	InceptionV3	Unsupervised	44.07	40.6	53.9
DR-DSN [81]	KTS	GoogLeNet	Unsupervised	2.63	41.4	57.6
SMIL [28]	KTS	VGGNet-16	Unsupervised	-	41.2	51.3
M-AVS [24]	KTS	GoogLeNet	Supervised	4.40	44.4	61.0
UnpairedVSN [46]	KTS	GoogLeNet	Unsupervised	-	47.5	55.6
FCSN [47]	KTS	GoogLeNet	Supervised	-	48.8	58.4
CSNet [25]	KTS	GoogLeNet	Unsupervised	-	51.3	58.8
Cycle-SUM [68]	KTS	GoogLeNet	Unsupervised	-	41.9	57.6
VSST [80]	KTS	VGGNet-16	Supervised	-	53.4	62.0
VASNet [12]	KTS	GoogLeNet	Supervised	7.35	49.7	61.4
RR-STG_{gen}	KTS	ResNet-50	Supervised	13.02	48.7	62.1
RR-STG	KTS	ResNet-50	Supervised	13.02	53.4	63.0

networks (SASUM) [78], deep reinforcement learning for unsupervised video summarization (DR-DSN) [41], action parsing driven video summarization (SMIL) [79], video summarization with attention-based encoder-decoder networks (M-AVS) [23], video summarization by learning from unpaired data (UnpairedVSN) [43], video summarization using fully convolutional sequence networks (FCSN) [80], discriminative feature learning for unsupervised video summarization (CSNet) [42], cycle-consistent adversarial lstm networks (Cycle-SUM) [44], video summarization via spatio-temporal deep architecture (VSST) [46], summarizing videos with attention (VASNet) [25]. Note that we also compared our method with the supervised versions of DR-DSN and SUM-GAN and a modification version of AVS (M-AVS).

B. Results and Analyses

In this subsection, we conducted comparisons with state-of-the-art video summarization methods under three dataset settings and provided comparisons with graph pooling methods on video summarization and graph classification datasets.

1) *Comparisons With State-of-the-Art Video Summarization Methods:* We provided comparisons between the proposed method with state-of-the-art non-deep and deep learning based methods. Table II tabulates experimental results on the SumMe and TVSum datasets by using the F-score (%) measure. Moreover, we summarized the main properties of different methods, including shot detection strategies, feature extraction methods, learning paradigms, and model parameters. We observe that our method significantly outperforms state-of-the-art non-deep and deep learning based video summarization methods except VSST [46] under the canonical setting. Our

method achieves 53.4 % accuracy on SumMe and 63.0 % accuracy on TVSum, which obtains 1.6% improvements over the previous state-of-the-art on TVSum. However, our method with GCN layers only achieves a comparable performance with the other methods and is even 4.7% and 0.9 % behind the performance of our method with SAEPool on the SumMe and TVSum datasets, respectively. The reason may be that the self-attention edge pooling module can flexibly drop erroneous relations.

2) *Comparisons Under the Augmented and Transfer Settings:* We also compared our method with state-of-the-art methods under augmented and transfer settings. We conducted the augmented and transfer experiments to evaluate the model scalability and verify the knowledge transferability of different methods across other video domains.

Table III presents experimental results of different methods under the augmented and transfer settings on the SumMe and TVSum datasets. We observe that our method consistently outperforms state-of-the-art video summarization methods on both datasets. The model with GCN compares favorably to the others, especially on the TVSum dataset. Under the augmented setting, our method achieves 54.8% accuracy and 63.6% accuracy on the SumMe and TVSum datasets, which are 2.7% and 1.2% relative improvements against the previous state-of-the-art. Meanwhile, our method achieves 45.4% accuracy and 59.7% accuracy on the SumMe and TVSum datasets under the transfer setting. Our method shows an encouraging generalization ability compared with the other methods. We speculate that since our method adaptively extracts object proposals and formulates these object proposals as spatial and temporal graphs, this process is agnostic to video sequences. Moreover, our method attains the best performance under the augmented

TABLE III

COMPARISONS OF VIDEO SUMMARIZATION METHODS ON SUMME AND TVSUM UNDER THE AUGMENTED AND TRANSFER SETTINGS

Method	Augmented		Transfer	
	SumMe	TVSum	SumMe	TVSum
vsLSTM [72]	41.6	57.9	40.7	56.9
dppLSTM [72]	42.9	59.6	41.8	58.7
CSNet [25]	52.1	59.0	45.1	59.2
SUM-GAN _{dpp} [35]	43.4	59.5	-	-
SUM-GAN _{sup} [35]	43.6	61.2	-	-
DR-DSN [81]	42.8	58.4	42.4	57.8
DR-DSN _{sup} [81]	43.9	59.8	42.6	58.9
A-AVS [24]	44.6	60.8	-	-
M-AVS [24]	46.1	61.8	-	-
UnpairedVSN [46]	-	-	41.6	55.7
FCSN [47]	50.2	59.1	45.0	57.4
VASNet [12]	51.1	62.4	-	-
RR-STG_{gcn}	50.3	63.5	44.9	59.1
RR-STG	54.8	63.6	45.4	59.7

setting compared with results under the canonical and transfer settings, proving its good model scalability.

3) *Comparisons With State-of-the-Art Graph Pooling Methods*: To further validate the superiority of the SAEPool layer, we conducted comparisons on the SumMe and TVSum datasets by only replacing the SAEPool layer with other graph pooling layers. Moreover, for a fair comparison, we set the pooling rates of different graph pooling layers to be equal, which ensures that the same number of relations in a graph was dropped. Compared methods include sequence to sequence for sets (Set2Set) [82], an end-to-end deep learning architecture for graph classification (SortPool) [64], and self-attention graph pooling (SAGPool) [63].

Table IV presents experimental results of different graph pooling layers on the SumMe and TVSum datasets with the F-score (%) measure. We observe that the SAEPool layer achieves the best performance, exceeding the other layers by 0.9% on the SumMe dataset and by 0.9% on the TVSum dataset. Without an edge pooling layer, the GCN model obtains the worst result on the SumMe dataset. The SAGPool layer is a special case of the SAEPool layer that improves the performance of the SAGPool layer by about 2.2% and 1.3% on the SumMe and TVSum datasets. The reason is that compared with the other graph pooling methods, SAEPool leverages not only the graph topology and graph representations but also the second-order information for edge pooling.

4) *Comparisons With Graph Pooling Methods on Graph Classification*: As previous graph pooling methods were evaluated in the graph classification task, we also conducted experiments on graph classification datasets to validate the effectiveness of the SAEPool layer. Following experimental settings in [63], we used three standard benchmark datasets, including PROTEINS [83], NCI1 [84], and NCI109 [84]. Each dataset contains more than 1K graphs. PROTEINS [83] is a protein dataset with nodes being secondary structure elements. NCI [84] contains a set of chemical compounds, where the

TABLE IV

COMPARISONS (%) OF DIFFERENT GRAPH POOLING METHODS ON THE SUMME AND TVSUM DATASETS

Method	SumMe	TVSum
Set2Set [56]	51.8	59.5
SortPool [74]	52.5	59.6
SAGPool [27]	51.2	61.7
GCN	48.7	62.1
SAEPool	53.4	63.0

TABLE V

DESCRIPTIONS OF GRAPH CLASSIFICATION DATASETS

Dataset	#Graphs	#Nodes per Graph	#Edges per Graph
PROTEINS [15]	1113	39.06	72.82
NCI1 [53]	4110	29.87	32.30
NCI109 [6]	4127	29.68	32.13

nodes and edges are atoms and chemical bonds. Among them, NCI1 and NCI109 are widely used in graph classification. Table V summarized key properties of these graph classification datasets, including the graph number, nodes, and edges per graph. Specifically, we adopted the global pooling and hierarchical pooling architectures, which are the same as the architecture settings in [63].

Table VI shows the average accuracies of different graph pooling methods on the PROTEINS, NCI1, and NCI109 datasets by using global and hierarchical architectures. We observe that our graph pooling method yields superior performance compared with state-of-the-art graph pooling methods, achieving about 9.4%, 0.3%, 1.4% improvements with the global architecture, while about 9.5%, 4.8%, and 8.0% improvements with the hierarchical architecture. Our method largely improves the performance of the other methods on the PROTEINS dataset. We speculate that since in this dataset each node has about twice the edges on average, graph modeling approaches severely suffer from the graph edge saturation problem, but our method is more flexible to remove unconcerned edges.

C. Ablation Studies

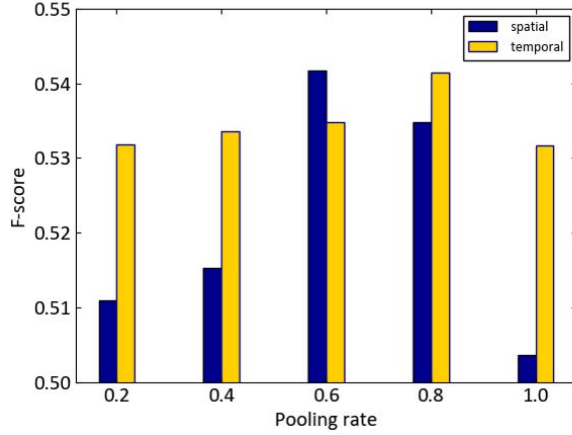
In this subsection, we conducted ablation studies on embedding parameters, pooling rates, and the number of SAEPool layers. Then, we analyzed the ability of redundancy reduction.

1) *Effect of Embedding Parameters*: To investigate the influence of the embedding parameters Θ_1 and Θ_2 in Eq. (10) and Eq. (11), we conducted experiments on the SumMe and TVSum datasets over spatial and temporal graphs. Table VII presents experimental results with different parameter settings in the SAEPool layer. We observe that the SAEPool layer achieves comparable performance under different parameter settings on the SumMe and TVSum datasets. To be simple, we set the embedding parameters of the SAEPool layer to be equal in default.

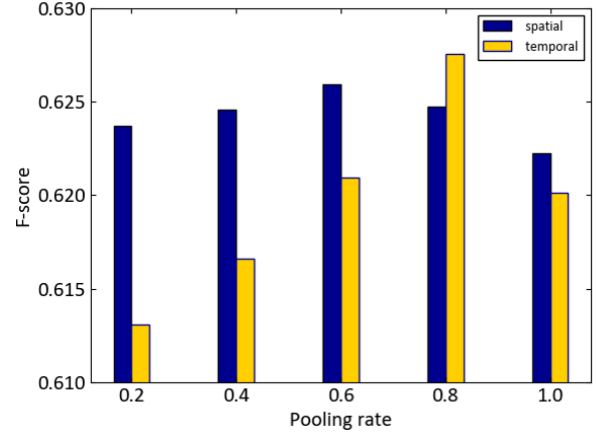
2) *Effect of Pooling Rates*: To evaluate the influence of pooling rates in the SAEPool module, we provided

TABLE VI
AVERAGE ACCURACY (%) OF DIFFERENT GRAPH POOLING METHODS ON THREE GRAPH CLASSIFICATION DATASETS BY USING THE GLOBAL POOLING ARCHITECTURE AND THE HIERARCHICAL POOLING ARCHITECTURE

Method	Global pooling			Hierarchical pooling		
	PROTEINS	NCI1	NCI109	PROTEINS	NCI1	NCI109
Set2Set [56]	66.06	68.55	69.78	68.20	62.32	61.98
SortPool [74]	66.72	73.82	74.02	71.10	67.02	66.12
SAGPool [27]	70.04	74.18	74.06	71.86	67.45	67.86
SAEPool	79.46	74.48	75.50	80.36	72.26	75.85



(a) the SumMe dataset



(b) the TVSum dataset

Fig. 5. F-score results of different pooling rates in the SAEPool module. The blue (yellow) bar labeled as spatial (temporal) indicates setting temporal (spatial) rates fixed and changing the spatial (temporal) pooling rates.

TABLE VII

ABLATION STUDIES OF THE EMBEDDING PARAMETERS Θ_1 AND Θ_2 OVER SPATIAL AND TEMPORAL GRAPHS. $\times$ INDICATES THE EMBEDDING LAYERS WITH DIFFERENT PARAMETERS, $\checkmark$ INDICATES THE EMBEDDING LAYERS WITH THE SAME PARAMETERS

Dataset	Spatial	Temporal	F-score
SumMe	$\times$	$\times$	53.6
	$\times$	$\checkmark$	52.4
	$\checkmark$	$\times$	54.1
	$\checkmark$	$\checkmark$	53.4
TVSum	$\times$	$\times$	61.5
	$\times$	$\checkmark$	62.6
	$\checkmark$	$\times$	61.4
	$\checkmark$	$\checkmark$	63.0

TABLE VIII

ABLATION STUDIES OF THE NUMBER OF STACKED LAYERS OVER SPATIAL AND TEMPORAL GRAPHS. $\times$ INDICATES THAT ONE SINGLE LAYER WAS APPLIED, WHILE $\checkmark$ INDICATES TWO LAYERS WERE STACKED

Dataset	Spatial	Temporal	F-score
SumMe	$\times$	$\times$	49.9
	$\times$	$\checkmark$	50.2
	$\checkmark$	$\times$	52.2
	$\checkmark$	$\checkmark$	53.4
TVSum	$\times$	$\times$	62.0
	$\times$	$\checkmark$	62.1
	$\checkmark$	$\times$	61.9
	$\checkmark$	$\checkmark$	63.0

comparisons over spatial and temporal graphs. Note that we fixed the pooling rate of spatial (temporal) graphs when changing the pooling rate of temporal (spatial) graphs. We investigated the effect by setting pooling rates in temporal and spatial domains as 0.8 and 0.6 on SumMe while 0.9 and 0.3 on TVSum. Fig. 5 depicts experimental results of different pooling rates on the SumMe dataset (Fig. 5(a)) and on the TVSum dataset (Fig. 5(b)). We observe that the pooling rate in the SAEPool module has a significant influence on the final results and both the lower and higher pooling rates lead to performance degradation. The reason may be that a high pooling rate in graph modeling does harm to message passing

due to relation clutters, while a low pooling rate loses too much information, which also decreases performance. Moreover, since there are some detected object proposals of low quality, the pooling rate in the spatial domain should be relatively lower than the one in the temporal domain.

3) *Effect of Number of SAEPool Layers*: To analyze the number of stacked SAEPool layers, we conducted experiments on the SumMe and TVSum datasets. Table VIII presents experimental results with one or two SAEPool layers. We observe that our method obtains the best results with two stacked SAEPool layers on the SumMe and TVSum datasets, which are 3.5% and 1.0 % improvements compared with the

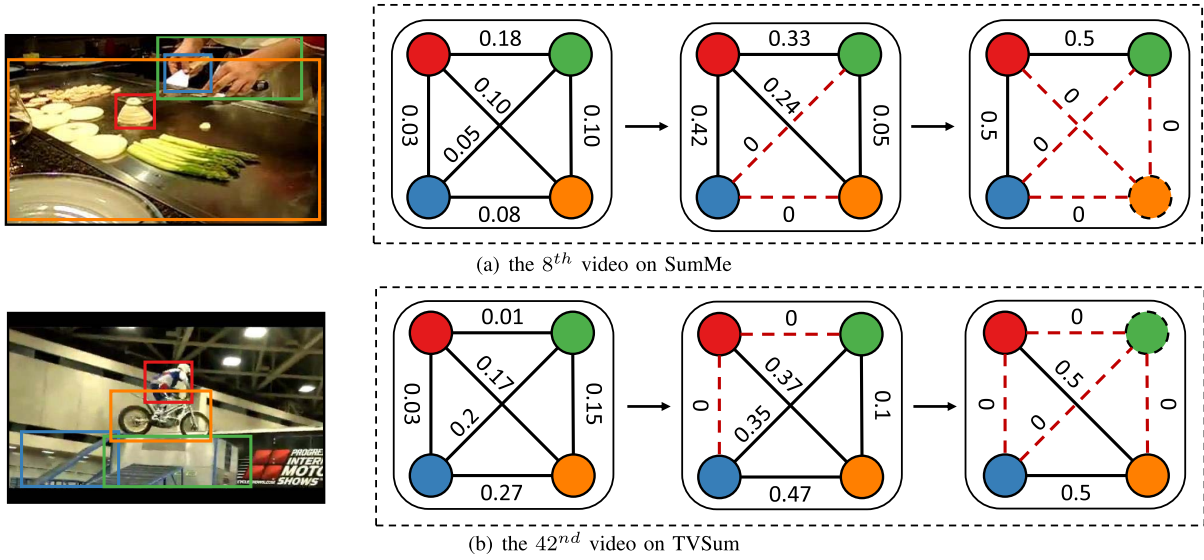


Fig. 6. The visualization of spatial relations by using example frames from the 8th video on SumMe (Fig. 6(a)) and the 42nd video on TVSum (Fig. 6(b)). Nodes represent object proposals and edges represent their relations. The object color in images corresponds to the node color in graphs. As each frame passes two SAEPool layers, there are three graphs, including the original graph, the first and second pooled graphs. Red dash lines indicate the removed edges.

TABLE IX

ABLATION STUDIES OF DIFFERENT LAYER SETTINGS OVER SPATIAL AND TEMPORAL GRAPHS. ✗ INDICATES THAT GRAPH CONVOLUTIONAL LAYER, WHILE ✓ INDICATES THE SAEPOOL LAYER

Dataset	Spatial	Temporal	F-score
SumMe	✗	✗	48.7
	✗	✓	53.2
	✓	✗	51.4
	✓	✓	53.4
TVSum	✗	✗	62.1
	✗	✓	62.2
	✓	✗	62.4
	✓	✓	63.0

TABLE X

F-SCORE (%) OF THE SAEPOOL AND GCN LAYERS BY USING DIFFERENT SETTINGS. FIRST AND SECOND INDICATE THAT WE USED MULTI-HOP SIMILARITIES IN THE FIRST AND SECOND SAEPOOL (OR GCN) LAYERS. AND BOTH MEANS THAT WE USED MULTI-HOP SIMILARITIES IN BOTH THE FIRST AND SECOND SAEPOOL LAYERS

Layers		First	Second	Both
GCN	Multi-hop	58.0	62.0	56.3
	One-hop	62.1	62.1	62.1
	Difference	4.1	0.1	5.8
SAEPool	Multi-hop	61.5	62.3	60.4
	One-hop	63.0	63.0	63.0
	Difference	1.5	0.7	2.6

only one layer setting. We both stacked two SAEPool layers over spatial and temporal graphs as the default setting.

4) *Effect of Different Layers*: To investigate the influence of different layer settings, we provided comparisons on the SumMe and TVSum datasets. Note that we simply replaced a graph convolutional layer with a SAEPool layer in the

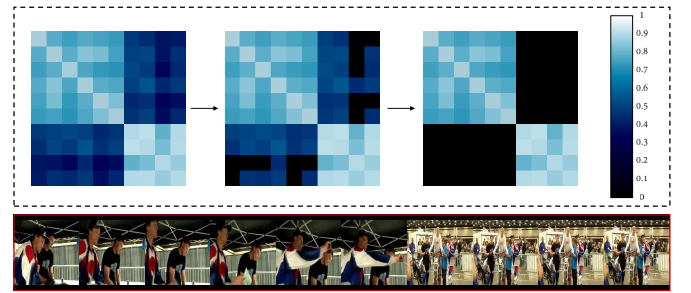


Fig. 7. The visualization of temporal relations by using an example snippet sampled from the 42nd video on TVSum. This snippet is about five seconds and totally has 150 video frames, among which we uniformly selected 10 frames. The above diagram presents heat maps of the adjacent matrices in the original graph, the first and second SAEPool layers. The figure below shows sampled 10 frames.

spatial or temporal domain. Table IX shows F-score results with different layer settings. We observe that our default model, applying the SAEPool layer in both spatial and temporal domains, achieves the best performance, and the model both with graph convolutional layers obtains the worst results. Experimental results provide favorable evidence to demonstrate the effectiveness of the proposed SAEPool layer compared with the graph convolutional layer.

5) *Ability of Redundance Reduction*: We investigated the effect of multi-hop similarities on the TVSum dataset. We set the adjacent matrix $\mathbf{A} = \mathbf{A} + \mathbf{A}^2$ in the SAEPool and GCN layers for comparisons. While dense connections are adopted in graphs to ensure that each node is linked, multi-hop connections provide higher-order connectivity. We replaced the adjacent matrix with these high-order similarities on spatial and temporal graphs. Table X presents experimental results with multi-hop similarities under different settings. We observe that the SAEPool and GCN layers with multi-hop similarities

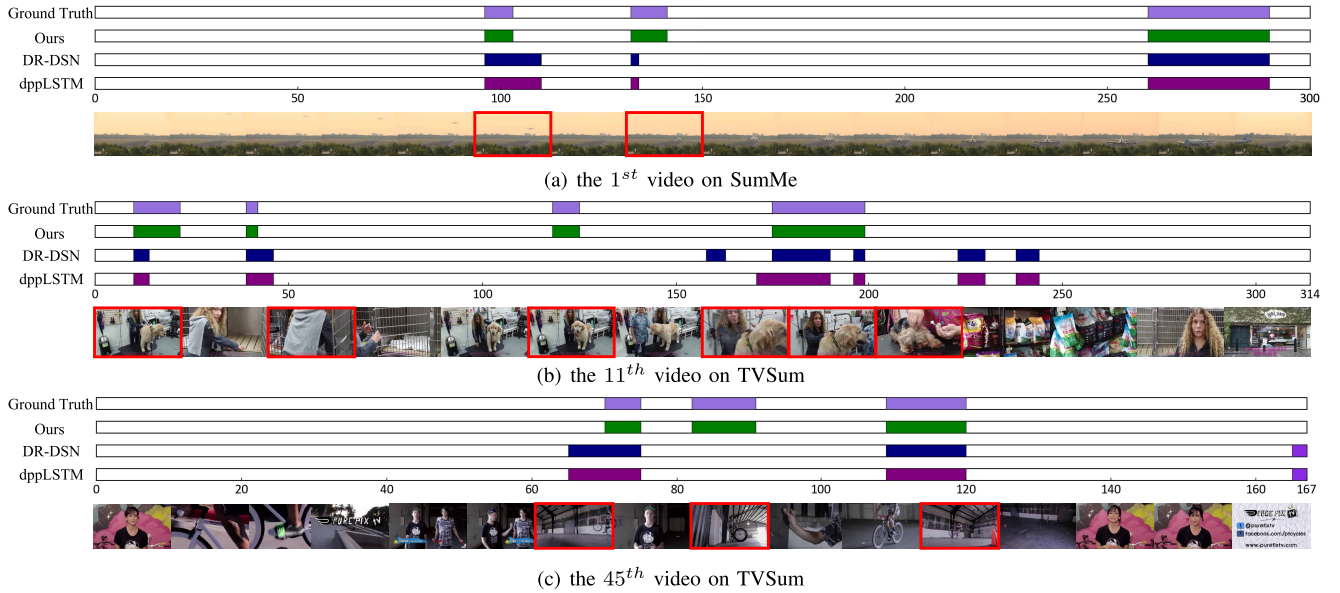


Fig. 8. The visualization of generated video summaries by using our method, DR-DSN [41], dppLSTM [38], and ground truth on the 1st video of SumMe and 11th and 45th videos of TVSum. The x-axis denotes the frame index and line segments along the x-axis represent selected shots. The below images are sampled from the corresponding video, where the selected images are marked with red boxes.

both achieve inferior results on the TVSum dataset. The reason may be that as the affine matrix A is constructed with erroneous relations, multi-hop connections provide more redundant information. However, our SAEPool has a good ability to eliminate the effect of redundancy compared with GCN, because the difference between the performance without multi-hop similarities and the performance with multi-hop similarities of SAEPool is smaller than that of GCN.

D. Time Analysis

We provided time analyses of compared methods, including vsLSTM and VASNet. Table XI shows the running time of compared methods on the SumMe and TVSum datasets. We observe that since vsLSTM, VASNet, and our method use KTS and the knapsack algorithm for shot segmentation and selection, these methods have the same running time in the key shot selection step. vsLSTM and VASNet both employ GoogLeNet for feature extraction, they cost 2.4 and 4.65 seconds on the SumMe and TVSum datasets. Our method applies Faster-RCNN to extract object proposals. Since Faster-RCNN is a two-stage approach, our method consumes too much time in the feature extraction step. However, the preprocessing stage can be handled in advance. In the representation learning step, our method exploits graph convolutional layers or self-attention edge pooling layers to conduct message passing for frame-level importance score prediction, which consumes less time than vsLSTM and more time than VASNet.

E. Visualization

In this subsection, we first provided the visualization of spatial relations. Then, we presented temporal heat maps. Finally, we visualized summary results.

1) *Visualization of Spatial Relations*: Fig. 6 visualizes spatial relations of example frames from the 8th video on SumMe (Fig. 6(a)) and the 42nd video on TVSum (Fig. 6(b)). Fig. 6(a) is about cooking food. Our method accurately detects the food, tool, action including hand and tool, and environment. All nodes are linked in the original graph. As the boundary box of the cooking action contains the boundary box of the tool, the connection between the cooking action and the tool is redundant. Moreover, the connections between the whole environment with the other three object proposals are also meaningless. Therefore, in the first SAEPool layer, our method removes unimportant linkings, e.g., connections between the tool and environment and between the tool and action. In the second layer, our method drops connections between the food and environment, the action and environment. Finally, there remain two relations among the food and tool, the food and action. Fig. 6(b) is about motorcycle stunt riding. There are also four detected proposals including 1) motorcyclist, 2) motorbike, 3) track, and 4) background. In the first pooling layer, our method removes connections between the background and motorcyclist, the motorcyclist and track. In the second pooling layer, our method further drops the connections between the background with the track and motorbike. Our method only keeps connections between the motorcyclist and motorbike, the motorbike and track. We observe that our SAEPool layer can precisely remove redundancy relations in the spatial domain.

2) *Visualization of Temporal Heat Maps*: It is really hard to clearly show dynamic relations and changes in the temporal domain. Therefore, we only sampled a snippet about five seconds from the 42nd video on the TVSum dataset, and we visualized heat maps that represent their relations among the sampled video frames. Fig. 7 presents the visualization of temporal relations. We observe that there are two segments,

TABLE XI
THE RUNNING TIME (SECOND) OF COMPARED METHODS ON THE SUMME AND TVSUM DATASETS

Dataset	Method	Feature extraction (pre-processing)	Representation learning (importance score prediction)	Key shot selection (post-processing)	
				Shot detection	Knapsack algorithm
SumMe	vsLSTM	2.40	0.12	4.72	0.01
	VASNet	2.40	0.01	4.72	0.01
	RR-STG	19.38	0.08	4.72	0.01
TVSum	vsLSTM	4.65	0.38	30.37	0.04
	VASNet	4.65	0.02	30.37	0.04
	RR-STG	31.38	0.25	30.37	0.04



Fig. 9. The visualization of failure cases on the SumMe and TVSum datasets. The first and second lines present the failure examples on the SumMe and TVSum datasets, respectively. The boundary boxes show the detected objects.

where the first segment includes six frames that are about a motorcyclist and the second segment contains the rest four frames that are about a group photo. As these two segments have different semantic topics, there are few connections. Therefore, our method removes all connections among these two segments after passing two SAEPool layers. These visualization results are intuitive evidence to prove the effectiveness of the SAEPool layer in the temporal domain.

3) *Visualization of Video Summaries*: We visualized example video summaries of our method and compared methods, including DR-DSN, dppLSTM, and ground truth, to qualitatively evaluate the quality of the generated summaries. The visualized example videos are the 1st video on SumMe and 11th and 45th videos on TVSum. The first video is concerned with an airplane landing at the airport, the second video tells about a pet store, and the third video is about how to keep balance on the bicycle with the rear wheel. Fig. 8 presents qualitative results of different video summarization methods. We clearly observe that our method intuitively outperforms DR-DSN and dppLSTM and produces video summaries that have high overlaps with ground truth summaries. Furthermore, while the 1st and 45th videos on the SumMe and TVSum datasets have few key shots of long duration and the 11th video on TVSum contains many key shots of short duration, our method accurately selected these key shots in summaries.

4) *Failure Examples*: We also presented some failure examples on the SumMe and TVSum datasets. Fig. 9 shows the missing detection cases in the object detector. We can observe that without the fine-tuning process of detection on the video summarization datasets, some unusual objects are undetected

including the large lake, confusing bear, castle, person, and motorbike.

V. CONCLUSION

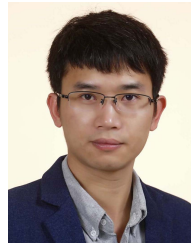
In this paper, we have presented a video summarization method called RR-STG to extract spatial-temporal representations via spatial-temporal graphs. Our method detects object proposals in each video frame and formulates these object proposals as a spatial graph. To further capture temporal dependencies, we aggregate spatial information and model video frames via a temporal graph. Then, we employ graph convolution layers or the proposed self-attention edge pooling layers to perform message passing over spatial and temporal graphs. In future work, we are interested in fusing auxiliary information, such as audio, text, and other metadata, to help build a spatial-temporal graph for video summarization.

REFERENCES

- [1] C. T. Dang and H. Radha, "Heterogeneity image patch index and its application to consumer video summarization," *IEEE Trans. Image Process.*, vol. 23, no. 6, pp. 2704–2718, Jun. 2014.
- [2] K. Muhammad, T. Hussain, M. Tanveer, G. Sannino, and V. H. C. de Albuquerque, "Cost-effective video summarization using deep CNN with hierarchical weighted fusion for IoT surveillance networks," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4455–4463, May 2020.
- [3] K. Muhammad, T. Hussain, J. D. Ser, V. Palade, and V. H. C. de Albuquerque, "DeepReS: A deep learning-based video summarization strategy for resource-constrained industrial surveillance scenarios," *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 5938–5947, Sep. 2020.
- [4] R. Panda, N. C. Mithun, and A. K. Roy-Chowdhury, "Diversity-aware multi-video summarization," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4712–4724, Oct. 2017.
- [5] E. Elhamifar, G. Sapiro, and R. Vidal, "See all by looking at a few: Sparse modeling for finding representative objects," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2012, pp. 1600–1607.
- [6] E. Elhamifar, G. Sapiro, and S. Sastry, "Dissimilarity-based sparse subset selection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 11, pp. 2182–2197, Nov. 2016.
- [7] D. Potapov, M. Douze, Z. Harchaoui, and C. Schmid, "Category-specific video summarization," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2014, pp. 540–555.
- [8] S. E. F. de Avila, A. P. B. Lopes, A. da Luz, Jr., and A. de Albuquerque Araújo, "VSUMM: A mechanism designed to produce static video summaries and a novel evaluation method," *Pattern Recognit. Lett.*, vol. 32, no. 1, pp. 56–68, Jan. 2011.
- [9] R. Panda and A. Roy-Chowdhury, "Multi-view surveillance video summarization via joint embedding and sparse optimization," *IEEE Trans. Multimedia*, vol. 19, no. 9, pp. 2010–2021, Sep. 2017.
- [10] S. Zhang, Y. Zhu, and A. K. Roy-Chowdhury, "Context-aware surveillance video summarization," *IEEE Trans. Image Process.*, vol. 25, no. 11, pp. 5469–5478, Nov. 2016.

- [11] E. Elhamifar and M. C. D. P. Kaluza, "Online summarization via submodular and convex optimization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1818–1826.
- [12] K. Muhammad, T. Hussain, and S. L. Baik, "Efficient CNN based summarization of surveillance videos for resource-constrained devices," *Pattern Recognit. Lett.*, vol. 130, pp. 370–375, Feb. 2020.
- [13] J. Wu, S.-H. Zhong, J. Jiang, and Y. Yang, "A novel clustering method for static video summarization," *Multimedia Tools Appl.*, vol. 76, no. 7, pp. 9625–9641, Apr. 2017.
- [14] B. Gong, W. L. Chao, K. L. Grauman, and F. Sha, "Diverse sequential subset selection for supervised video summarization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2069–2077.
- [15] K. Zhou, T. Xiang, and A. Cavallaro, "Video summarisation by classification with deep reinforcement learning," in *Proc. Brit. Mach. Vis. Conf.*, 2018, pp. 1–13.
- [16] T. Yao, T. Mei, and Y. Rui, "Highlight detection with pairwise deep ranking for first-person video summarization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 982–990.
- [17] S. Huang, X. Li, Z. Zhang, F. Wu, and J. Han, "User-ranking video summarization with multi-stage spatio-temporal representation," *IEEE Trans. Image Process.*, vol. 28, no. 6, pp. 2654–2664, Jun. 2019.
- [18] T. Hussain, K. Muhammad, W. Ding, J. Lloret, S. W. Baik, and V. H. C. de Albuquerque, "A comprehensive survey of multi-view video summarization," *Pattern Recognit.*, vol. 109, pp. 1–15, Jan. 2020.
- [19] T. Hussain, K. Muhammad, A. Ullah, Z. Cao, S. W. Baik, and V. H. C. de Albuquerque, "Cloud-assisted multiview video summarization using CNN and bidirectional LSTM," *IEEE Trans. Ind. Informat.*, vol. 16, no. 1, pp. 77–86, Jan. 2020.
- [20] B. Zhao, X. Li, and X. Lu, "Hierarchical recurrent neural network for video summarization," in *Proc. 25th ACM Int. Conf. Multimedia*, Oct. 2017, pp. 863–871.
- [21] Y. Zhang, M. Kampffmeyer, X. Liang, D. Zhang, M. Tan, and E. P. Xing, "Dilated temporal relational adversarial network for generic video summarization," *Multimedia Tools Appl.*, vol. 78, no. 24, pp. 35237–35261, Dec. 2019.
- [22] B. Mahasseni, M. Lam, and S. Todorovic, "Unsupervised video summarization with adversarial LSTM networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 202–211.
- [23] Z. Ji, K. Xiong, Y. Pang, and X. Li, "Video summarization with attention-based encoder-decoder networks," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 6, pp. 1709–1717, Jun. 2020.
- [24] Y.-T. Liu, Y.-J. Li, F.-E. Yang, S.-F. Chen, and Y.-C. F. Wang, "Learning hierarchical self-attention for video summarization," in *Proc. IEEE Int. Conf. Image Process.*, Sep. 2019, pp. 3377–3381.
- [25] J. Fajtl, H. S. Sokeh, V. Argyriou, D. Monekosso, and P. Remagnino, "Summarizing videos with attention," in *Proc. Asian Conf. Comput. Vis.*, 2018, pp. 39–54.
- [26] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.
- [27] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
- [28] J. Wu, S.-H. Zhong, and Y. Liu, "MvsGCN: A novel graph convolutional network for multi-video summarization," in *Proc. 27th ACM Int. Conf. Multimedia*, Oct. 2019, pp. 827–835.
- [29] J. Wu, S.-H. Zhong, Z. Ma, S. J. Heinen, and J. Jiang, "Gaze aware deep learning model for video summarization," in *Proc. Pacific Rim Conf. Multimedia*, 2018, pp. 285–295.
- [30] T. Hussain *et al.*, "Multi-view summarization and activity recognition meet edge computing in IoT environments," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9634–9644, Jun. 2020.
- [31] Y. Wu, O. E. F. Bourahla, X. Li, F. Wu, Q. Tian, and X. Zhou, "Adaptive graph representation learning for video person re-identification," *IEEE Trans. Image Process.*, vol. 29, pp. 8821–8830, 2020.
- [32] J. Zhang, F. Shen, X. Xu, and H. T. Shen, "Temporal reasoning graph for activity recognition," *IEEE Trans. Image Process.*, vol. 29, pp. 5491–5506, 2020.
- [33] Y. Shen, C. Feng, Y. Yang, and D. Tian, "Mining point cloud local structures by kernel correlation and graph pooling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4548–4557.
- [34] X. Li, H. Li, and Y. Dong, "Meta learning for task-driven video summarization," *IEEE Trans. Ind. Electron.*, vol. 67, no. 7, pp. 5778–5786, Jul. 2020.
- [35] W. Zhu, J. Lu, J. Li, and J. Zhou, "DSNet: A flexible detect-to-summarize network for video summarization," *IEEE Trans. Image Process.*, vol. 30, pp. 948–962, 2021.
- [36] M. Otani, Y. Nakashima, E. Rahtu, and J. Heikkilä, "Rethinking the evaluation of video summaries," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7596–7604.
- [37] J. Wu, S.-H. Zhong, and Y. Liu, "Dynamic graph convolutional network for multi-video summarization," *Pattern Recognit.*, vol. 107, pp. 1–13, Nov. 2020.
- [38] K. Zhang, W. Chao, F. Sha, and K. Grauman, "Video summarization with long short-term memory," in *Proc. Eur. Conf. Comput. Vis.*, Oct. 2016, pp. 766–782.
- [39] B. Zhao, X. Li, and X. Lu, "TTH-RNN: Tensor-train hierarchical recurrent neural network for video summarization," *IEEE Trans. Ind. Electron.*, vol. 68, no. 4, pp. 3629–3637, Apr. 2021.
- [40] K. Zhang, K. Grauman, and F. Sha, "Retrospective encoders for video summarization," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 383–399.
- [41] K. Zhou, Y. Qiao, and T. Xiang, "Deep reinforcement learning for unsupervised video summarization with diversity-representativeness reward," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 7582–7589.
- [42] Y. Jung, D. Cho, D. Kim, S. Woo, and I. S. Kweon, "Discriminative feature learning for unsupervised video summarization," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 8537–8544.
- [43] M. Rochan and Y. Wang, "Video summarization by learning from unpaired data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7902–7911.
- [44] L. Yuan, F. E. Tay, P. Li, L. Zhou, and J. Feng, "Cycle-sum: Cycle-consistent adversarial LSTM networks for unsupervised video summarization," in *Proc. AAAI Conf. Artif. Intelligence.*, 2018, pp. 9144–9150.
- [45] S. Xiao, Z. Zhao, Z. Zhang, X. Yan, and M. Yang, "Convolutional hierarchical attention network for query-focused video summarization," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 12426–12433.
- [46] S.-H. Zhong, J. Wu, and J. Jiang, "Video summarization via spatio-temporal deep architecture," *Neurocomputing*, vol. 332, pp. 224–235, Mar. 2019.
- [47] P. Battaglia *et al.*, "Interaction networks for learning about objects, relations and physics," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4502–4510.
- [48] A. Santoro *et al.*, "A simple neural network module for relational reasoning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4967–4976.
- [49] K. Tang, H. Zhang, B. Wu, W. Luo, and W. Liu, "Learning to compose dynamic tree structures for visual contexts," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 6619–6628.
- [50] R. Zellers, M. Yatskar, S. Thomson, and Y. Choi, "Neural motifs: Scene graph parsing with global context," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5831–5840.
- [51] X. Chen, L.-J. Li, L. Fei-Fei, and A. Gupta, "Iterative visual reasoning beyond convolutions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7239–7248.
- [52] J. Shi, H. Zhang, and J. Li, "Explainable and explicit visual reasoning over scene graphs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8376–8384.
- [53] F. Baradel, N. Neverova, C. Wolf, J. Mille, and G. Mori, "Object level visual reasoning in videos," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 105–121.
- [54] R. Herzig *et al.*, "Spatio-temporal action graph networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 2347–2356.
- [55] X. Wang and A. Gupta, "Videos as space-time region graphs," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 399–417.
- [56] T. Zhuo, Z. Cheng, P. Zhang, Y. Wong, and M. Kankanhalli, "Explainable video action reasoning via prior knowledge and state transitions," in *Proc. 27th ACM Int. Conf. Multimedia*, Oct. 2019, pp. 521–529.
- [57] Y.-H.-H. Tsai, S. Divvala, L.-P. Morency, R. Salakhutdinov, and A. Farhadi, "Video relationship reasoning using gated spatio-temporal energy graph," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10424–10433.
- [58] Y. Jang, Y. Song, C. D. Kim, Y. Yu, Y. Kim, and G. Kim, "Video question answering with spatio-temporal reasoning," *Int. J. Comput. Vis.*, vol. 127, no. 10, pp. 1385–1412, Oct. 2019.
- [59] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3844–3852.

- [60] S. Rhee, S. Seo, and S. Kim, "Hybrid approach of relation network and localized graph convolutional filtering for breast cancer subtype classification," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, p. 3527.
- [61] M. Simonovsky and N. Komodakis, "Dynamic edge-conditioned filters in convolutional neural networks on graphs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 3693–3702.
- [62] H. Gao and S. Ji, "Graph U-Nets," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2083–2092.
- [63] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6661–6670.
- [64] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 4438–4810.
- [65] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4445–4800.
- [66] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Lió, "Towards sparse hierarchical graph classifiers," in *Proc. Adv. Neural Inf. Process. Syst. Workshop*, 2018, pp. 1–6.
- [67] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask R-CNN," in *Proc. ICCV*, Oct. 2017, pp. 2961–2969.
- [68] Y. Li *et al.*, "Attention-guided unified network for panoptic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7026–7035.
- [69] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent.*, 2015, pp. 1–14.
- [70] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [71] M. Gygli, H. Grabner, H. Riemenschneider, and L. Van Gool, "Creating summaries from user videos," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2014, pp. 505–520.
- [72] Y. Song, J. Vallmitjana, A. Stent, and A. Jaimes, "TVSum: Summarizing web videos using titles," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 5179–5187.
- [73] Y. Li and B. Merialdo, "Multi-video summarization based on video-MMR," in *Proc. Int. Workshop Image Anal. Multimedia Interact. Services*, 2010, pp. 1–4.
- [74] K. Zhang, W.-L. Chao, F. Sha, and K. Grauman, "Summary transfer: Exemplar-based subset selection for video summarization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1059–1067.
- [75] B. Zhao and E. P. Xing, "Quasi real-time summarization for consumer videos," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 2513–2520.
- [76] X. Li, B. Zhao, and X. Lu, "A general framework for edited video and raw video summarization," *IEEE Trans. Image Process.*, vol. 26, no. 8, pp. 3652–3664, Aug. 2017.
- [77] J. Meng, S. Wang, H. Wang, J. Yuan, and Y.-P. Tan, "Video summarization via multi-view representative selection," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 1189–1198.
- [78] H. Wei, B. Ni, Y. Yan, H. Yu, X. Yang, and C. Yao, "Video summarization via semantic attended networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 216–223.
- [79] J. Lei, Q. Luan, X. Song, X. Liu, D. Tao, and M. Song, "Action parsing-driven video summarization based on reinforcement learning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 29, no. 7, pp. 2126–2137, Jul. 2019.
- [80] M. Rochan, L. Ye, and Y. Wang, "Video summarization using fully convolutional sequence networks," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 347–363.
- [81] T.-Y. Lin *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2014, pp. 740–755.
- [82] O. Vinyals, S. Bengio, and M. Kudlur, "Order matters: Sequence to sequence for sets," in *Proc. Int. Conf. Learn. Represent.*, 2016, pp. 1–11.
- [83] P. D. Dobson and A. J. Doig, "Distinguishing enzyme structures from non-enzymes without alignments," *J. Mol. Biol.*, vol. 330, no. 4, pp. 771–783, Jul. 2003.
- [84] N. Wale, I. Watson, and G. Karypis, "Comparison of descriptor spaces for chemical compound retrieval and classification," *Knowl. Inf. Syst.*, vol. 14, no. 3, pp. 347–375, 2008.



Wencheng Zhu received the B.Eng. and M.Eng. degrees from Tianjin University, Tianjin, China, in 2014 and 2017, respectively, and the Ph.D. degree from the Department of Automation, Tsinghua University, Beijing, China, in 2021. Currently, he serves as a Postdoctoral Fellow with the Department of Hydraulic Engineering, Tsinghua University. His research interests include video summarization and video object segmentation.



Yucheng Han received the B.Eng. degree from the Department of Automation, Tsinghua University, Beijing, China, in 2021. His research interests include causal inference and computer vision.



Jiwen Lu (Senior Member, IEEE) received the B.Eng. degree in mechanical engineering and the M.Eng. degree in electrical engineering from the Xi'an University of Technology, Xi'an, China, in 2003 and 2006, respectively, and the Ph.D. degree in electrical engineering from Nanyang Technological University, Singapore, in 2012. He is currently an Associate Professor with the Department of Automation, Tsinghua University, Beijing, China. His current research interests include computer vision and pattern recognition. He was/is a member of the Image, Video and Multidimensional Signal Processing Technical Committee, the Multimedia Signal Processing Technical Committee, and the Information Forensics and Security Technical Committee of the IEEE Signal Processing Society; and the Multimedia Systems and Applications Technical Committee and the Visual Signal Processing and Communications Technical Committee of the IEEE Circuits and Systems Society. He is a fellow of IAPR. He was a recipient of the National Natural Science Funds for Distinguished Young Scholar. He serves as the General Co-Chair for the International Conference on Multimedia and Expo (ICME) 2022 and the Program Co-Chair for the International Conference on Multimedia and Expo 2020, the International Conference on Automatic Face and Gesture Recognition (FG) 2023, and the International Conference on Visual Communication and Image Processing (VCIP) 2022. He serves as the Co-Editor-in-Chief for *Pattern Recognition Letters* and an Associate Editor for the IEEE TRANSACTIONS ON IMAGE PROCESSING, the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY, the IEEE TRANSACTIONS ON BIOMETRICS, BEHAVIOR, AND IDENTITY SCIENCE, and *Pattern Recognition*.



Jie Zhou (Senior Member, IEEE) received the B.S. and M.S. degrees from the Department of Mathematics, Nankai University, Tianjin, China, in 1990 and 1992, respectively, and the Ph.D. degree from the Institute of Pattern Recognition and Artificial Intelligence, Huazhong University of Science and Technology (HUST), Wuhan, China, in 1995. From 1995 to 1997, he served as a Postdoctoral Fellow at the Department of Automation, Tsinghua University, Beijing, China, where he has been a Full Professor with the Department of Automation since 2003. In recent years, he has authored more than 300 papers in peer-reviewed journals and conferences. His research interests include computer vision, pattern recognition, and image processing. He is a fellow of IAPR. He was a recipient of the National Natural Science Funds for Distinguished Young Scholar.